CONTEXT SWITCHING

This is another very common OS interview topic. Understanding **why context switching happens** and **what it costs** is more important than memorizing a definition.

What is Context Switching?

A **context switch** is the process of **saving the state of the currently running process/thread and restoring the state of another process/thread**, so the CPU can continue executing the new one.

Why is Context Switching Needed?

Suppose your laptop has:

- Chrome
- Spotify
- VS Code

But only **one CPU core**.

Only one process can execute at any instant.

The OS rapidly switches between them:

```
Chrome
   ↓
VS Code
   ↓
Spotify
   ↓
Chrome
```

This creates the illusion that everything is running simultaneously.

What is "Context"?

The **context** is everything needed to resume execution later.

It includes:

- Program Counter (next instruction)
- CPU Registers
- Stack Pointer
- Process State
- Scheduling information

For processes, it also includes the **memory mapping (address space)**.

This information is stored in the **PCB** (or the thread's control block for threads).

Process Context Switch vs Thread Context Switch

Process Switch

Switching from:

```
Chrome

↓

VS Code
```

The OS must:

- Save registers
- Restore registers
- Switch address space (virtual memory)
- Switch page tables
- Potentially flush/reload TLB entries

- Update scheduling information

This is relatively expensive.

Thread Switch (Same Process)

Suppose Chrome has:

```
Chrome

UI Thread

↓

Network Thread
```

Both threads share:

- Memory
- Heap
- Files

The OS mainly switches:

- Registers
- Stack Pointer
- Program Counter

It **doesn't need to switch to a different address space**, making it much cheaper.

Why Does Context Switching Cost Time?

A context switch is **pure overhead**.

Imagine copying notes before changing tasks.

During the switch:

- The CPU isn't executing your application's instructions.
- It's busy saving and restoring state.

That time doesn't make your program finish any work.

Main Costs of Context Switching

1. CPU Time

The OS spends time:

- Saving registers
- Restoring registers
- Updating scheduling data

Instead of running user code.

---

2. Cache Pollution

The CPU cache contains data for Process A.

When switching to Process B:

```
Cache

Chrome Data
```

becomes

```
Cache

VS Code Data
```

When Chrome runs again, its cached data may no longer be there, leading to more cache misses and slower execution.

---

3. TLB Effects (Processes)

Modern CPUs use a **Translation Lookaside Buffer (TLB)** to speed up virtual-to-physical address translation.

Different processes have different address spaces.

When switching processes, many systems need to invalidate or update TLB entries.

Rebuilding these entries takes time.

(Modern CPUs often reduce this cost using techniques like ASIDs/PCIDs.)

---

4. Scheduler Overhead

The scheduler must decide:

- Which process runs next
- For how long
- Based on priorities and policies

This decision itself consumes CPU cycles.

---

5. Synchronization Cost (Threads)

Threads often need:

- Mutexes
- Semaphores
- Locks

Frequent switching between threads contending for locks can further reduce performance.

When Do Context Switches Occur?

A context switch can happen when:

- **Time slice expires** (preemptive multitasking)
- A process performs **blocking I/O**
- A higher-priority process becomes ready
- The current process voluntarily yields the CPU
- An interrupt occurs that causes the scheduler to run

Is Context Switching Always Bad?

No.

It's **necessary** for multitasking and responsiveness.

However:

- **Too many** context switches waste CPU time.
- **Too few** can make the system feel unresponsive.

Operating systems try to balance these trade-offs.

Interview Answer

> A **context switch** is the process of saving the execution state of the currently running process or thread and restoring the state of another so that execution can resume later. The saved state includes the program counter, CPU registers, stack pointer, and scheduling information. For processes, switching also involves changing the address space and memory mappings.

> Context switching has a performance cost because the CPU spends time saving and restoring state instead of executing application code. It can also reduce cache efficiency, incur TLB overhead during process switches, and add scheduler overhead. Thread context switches within the same process are generally cheaper than process context switches because threads share the same address space, so no memory mapping switch is required.

Quick Interview Tip

If asked:

> **"Why are thread context switches cheaper than process context switches?"**

Answer:

> Because threads within the same process share the same virtual address space, the operating system only needs to save and restore the thread's execution context (registers, stack pointer, and program counter). It doesn't need to switch page tables or change the process's address space, which makes thread context switches significantly less expensive than process context switches.